



Design and Implementation of a Security-Oriented CI/CD Pipeline for Practical Secure Software Engineering Education

Exposé

Eingereicht von: Yapa Puwakdandawage Ridma Kaveendra

Matrikelnummer: 1533868

Betreuende Professor: Prof. Dr. Michael Zohner

eingereicht am: 23.07.2026

Abstract

This thesis aims to develop a practical security-oriented CI/CD pipeline for Secure Software Engineering development practical exercises. The thesis will investigate and compare suitable security tools and integrate the selected tools into a working CI/CD environment covering static analysis, dynamic analysis using fuzzing, Software Bill of Materials (SBOM) generation and vulnerability management, third-party artefact verification, secure backend configuration, and secure Docker- and Kubernetes-based deployment. A financial application will also be developed as a reference application and the pipeline will be used to identify the security risks of the application. In addition, practical exercises and accompanying documentation will be developed to enable students to experience how different security mechanisms are applied throughout the software development lifecycle.

Contents

Abstract	ii
1 Introduction	1
2 State of the Art	2
3 Motivation and Objectives	3
4 Methodology	4
5 Work Plan and Timeline	5

1 Introduction

Modern software systems are increasingly developed as complex ecosystems involving application code, third-party dependencies, external software artefacts, container images, and infrastructure platforms. At the same time, automated CI/CD processes have become an important part of modern software development, enabling applications to be continuously built, tested, and deployed. This combination of complex software dependencies and automated development processes creates security challenges that lie not only in the application code but also extend beyond the code itself.

Security must therefore be considered throughout the software development and deployment lifecycle, including the analysis of source code and dependencies, the verification of software artefacts, the configuration of application and infrastructure components, and the security of the final deployment environment. Integrating such security activities into CI/CD workflows provides an opportunity to identify vulnerabilities earlier and to make security a continuous part of software development. This thesis addresses this topic in the context of practical Secure Software Engineering education by developing an environment in which students can experience security mechanisms within a realistic software development workflow.

2 State of the Art

DevSecOps approaches provide a broad range of technologies to integrate security activities into CI/CD processes. Existing tools address different aspects of software security, including tools such as SonarQube, Semgrep, OWASP, AFL++, OSV-Scanner, Grype, Sigstore, and Cosign. These technologies enable security checks to be automated and performed at different stages of the software development lifecycle. However, the individual tools generally focus on specific security tasks and differ considerably in their configuration, execution requirements, output formats, and reporting mechanisms.

The integration of several such technologies into a coherent CI/CD workflow therefore presents technical and organizational challenges. In addition to ensuring that the tools can operate within the same development environment, their results need to be interpreted and presented in a way that supports a consistent security workflow. This challenge is particularly relevant in an educational context, where the technical complexity of professional security tools must be balanced with the need for clear and understandable practical exercises. Consequently, the existing technologies provide the necessary building blocks for a comprehensive security-oriented CI/CD environment, but their appropriate combination and adaptation to a practical Secure Software Engineering learning environment requires further investigation.

3 Motivation and Objectives

The motivation for this thesis is the systematic selection, integration, and evaluation of existing security technologies in a realistic CI/CD environment, which students can use for learning purposes. Combined with exercises and documentation, this creates a setup that is suitable for teaching and allows students to experience how security tools interact within a complete development workflow. The project, therefore, focuses on identifying and combining suitable existing technologies in a way that balances technical functionality with practical usability in an educational setting.

The main objective is to design and implement a security-oriented CI/CD environment within the existing university Git and CI/CD infrastructure. The developed environment will cover static analysis, dynamic analysis using fuzzing frameworks, SBOM generation and vulnerability management, verification of third-party software artefacts, secure backend configuration, and secure Docker- and Kubernetes-based deployment. The work will investigate suitable tools, define an appropriate integration approach, and develop practical exercises and documentation that guide students through the resulting workflow. The final environment will be evaluated based on both its technical operation and its suitability for demonstrating Secure Software Engineering concepts in practice.

4 Methodology

The thesis will follow a design-and-development approach. First, a literature review will be conducted to establish the theoretical foundation and identify existing approaches, tools, and best practices for security-oriented CI/CD pipelines. Based on the results, suitable tools and integration strategies will be selected according to their technical capabilities and suitability for the intended educational environment. A realistic financial application will then be developed as a reference system for testing the security mechanisms. The selected tools will initially be implemented and validated in separate CI/CD pipelines, each addressing a specific security area. These pipelines will subsequently be integrated into a comprehensive security pipeline. The resulting environment will be tested using the reference application and evaluated with respect to its technical functionality, the successful integration of the security checks, and its suitability for practical Secure Software Engineering education. Finally, exercises and supporting documentation will be developed to enable students to apply the pipelines to their own financial application implementations.

5 Work Plan and Timeline

The thesis will be carried out over a period of approximately six months. The work will be organised into several overlapping phases, beginning with the literature review and continuing through application development, implementation and integration of the security pipelines, development of educational material, evaluation, and final thesis preparation. The planned timeline is shown in Table 5.1.

Table 5.1: Preliminary work plan and timeline

Weeks	Planned Task
Weeks 1–2	Literature review and analysis of existing approaches, tools, and best practices.
Weeks 3–6	Design and development of the financial reference application and investigation and selection of suitable security tools.
Weeks 7–15	Implementation and testing of individual CI/CD pipelines for the different security areas.
Weeks 16–17	Integration of the individual security pipelines into a comprehensive security pipeline.
Weeks 18–20	Development of practical exercises and supporting documentation.
Weeks 21–22	Evaluation of the complete environment and analysis of the results.
Weeks 1–24	Continuous thesis writing, revision, and finalisation.